

Biblioteca Noite Estrelada

Documentação Final

Tecnologias Utilizadas:

- Next.js 15 - framework utilizado para a construção do backend
- TypeScript - linguagem de programação, utilizada tanto na codificação do frontend quanto backend
- PostgreSQL 16 - Banco de dados relacional, usado para a persistência de dados no sistema
- Prisma ORM - Provê abstrações que facilitam a manipulação e interação do banco de dados
- Jest - framework para a criação de testes

Arquitetura Geral

Princípios e Padrões Aplicados

Princípios SOLID

1. Open/Closed Principle (OCP)

O sistema será desenvolvido de forma que novas funcionalidades possam ser adicionadas sem modificar código existente.

Este princípio pode ser observado nos Strategy para validação de mídias, onde a criação de novas validações é apenas possível pela extensão de uma interface com a assinatura do método com essa responsabilidade.

2. Liskov Substitution Principle (LSP)

Classes derivadas poderão substituir classes base sem alterar o comportamento esperado do sistema.

A hierarquia de Usuários, observada no arquivo schema.prisma da branch refatoração/remodelagem-de-leitor, temos as subclasses Funcionario e Leitor, que podem ser utilizadas no sistema onde classe de Usuários seja necessária, sem causar falhas no sistema.

3. Interface Segregation Principle (ISP)

Interfaces serão específicas para cada tipo de funcionalidade, evitando interfaces muito grandes.

Algumas interfaces DTOs deste sistema variam os campos de informações, de acordo com o contexto (criação e deleção, por exemplo).

4. Single Responsibility Principle (SRP)

Cada classe terá apenas uma responsabilidade específica. Este comportamento pode ser observado em diferentes classes/entidades do sistema:

- Routes: direcionamento das requisições provenientes das páginas, para o serviço requerido
- Services: Métodos voltado para os caso de uso e regras de negócio
- Repositories: Interação com banco de dados, partir do Prisma ORM

Padrões GRASP

1. Controller

O padrão **Controller** é utilizado para definir quais classes recebem e tratam as requisições do sistema.

- Este padrão foi utilizado nos Route Handlers, localizados nos arquivos route.ts, localizados na em /src/api. Possuem como principal função coordenar as requisições recebidas das páginas e enviá-las para o service necessário da operação

2. Information Expert

O padrão **Information Expert** determina que a responsabilidade deve ser atribuída à classe que possui as informações necessárias. Este padrão se encontra nos seguintes casos:

- Em `MidiaValidationStrategy`, cada operação possui as validações para cada tipo de mídia
- Na criação de `Exemplares`, onde cada `Mídia` possui uma lista destes, contendo as informações sobre cada e um conjunto de ações para edição e deleção

3. Low Coupling

O padrão **Low Coupling** busca reduzir dependências entre classes, facilitando a manutenção e reutilização do código.

- A separação do sistema em camadas permite que este se torne fracamente acoplado
- O uso de DTOs para restringir a informação entre a camada de negócios e apresentação

4. High Cohesion

- O padrão **High Cohesion** busca manter cada classe focada em uma responsabilidade específica.
- As classes de repositório e serviços em sua maioria se restringem a funções específicas. Nas leitores e mídias por exemplo, estas classes se concentram na criação, deleção, atualização, listagem e busca destas entidades.

Implementação do Sistema

CRUDS implementados

- Leitores: Listar, buscar , editar, deletar, atualizar
- Mídias: Listar, buscar , editar, deletar, atualizar
- Exemplares: Listar, buscar , editar, deletar, atualizar

Padrões GOF

O padrão Strategy foi implementado. O padrão Observer foi parcialmente implementado. O padrão Factory Method e Facade aparentemente foi implementados.

Strategy

Contexto de uso:

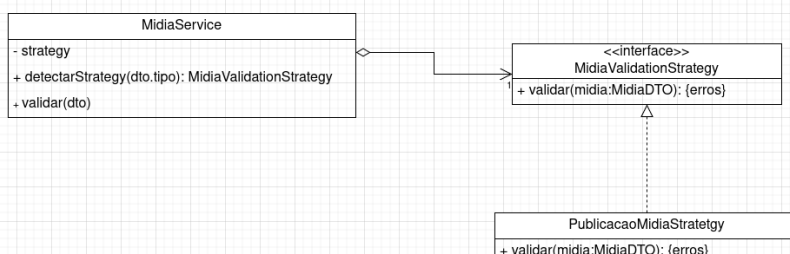
As regras de empréstimo podem variar de acordo com o tipo de Leitor associado ao Empréstimo. Por exemplo, um Aluno pode possuir prazo de empréstimo de sete dias, enquanto um Professor pode possuir prazo de trinta dias. Em vez de concentrar todas essas regras em uma única classe através de condicionais, cada política de empréstimo pode ser representada por uma estratégia específica responsável pelo cálculo do prazo, limite de exemplares simultâneos ou valor de multa.

Problema que resolve:

A necessidade de executar diferentes algoritmos para uma mesma operação dependendo do contexto em que ela é realizada.

Justificativa da escolha:

As regras de empréstimo representam comportamentos que podem variar independentemente do restante do sistema. Encapsular essas regras em estratégias separadas reduz o número de condicionais e facilita a inclusão de novas categorias de usuários sem necessidade de alterar a lógica principal do Empréstimo.



Factory Method

Contexto de uso:

A criação de entidades do sistema pode ser centralizada em fábricas especializadas ao invés de ser realizada diretamente pelos controladores ou interfaces gráficas. Por exemplo, ao cadastrar uma nova Publicação, o sistema pode decidir qual classe concreta instanciar conforme o tipo informado pelo usuário, como Livro, Revista ou Tese. Da mesma forma, o cadastro de usuários poderia criar diferentes subclasses de Leitor, caso o sistema possua categorias com comportamentos distintos, como Aluno de Graduação, Aluno de Pós-Graduação e Professor.

Problema que resolve:

A dependência direta de classes concretas por parte dos módulos que solicitam a criação dos objetos. Sem o padrão, cada local que cria uma entidade precisaria conhecer todas as classes concretas possíveis e as regras necessárias para decidir qual delas instanciar.

Justificativa da escolha:

A responsabilidade de decidir qual objeto criar passa a ficar concentrada em uma única abstração. Isso reduz o impacto de futuras alterações na hierarquia de entidades, já que novas subclasses podem ser adicionadas sem necessidade de modificar os módulos que utilizam essas entidades. O benefício é maior caso existam vários tipos de Publicação ou de Leitor com comportamentos próprios.

Observer

Contexto de uso:

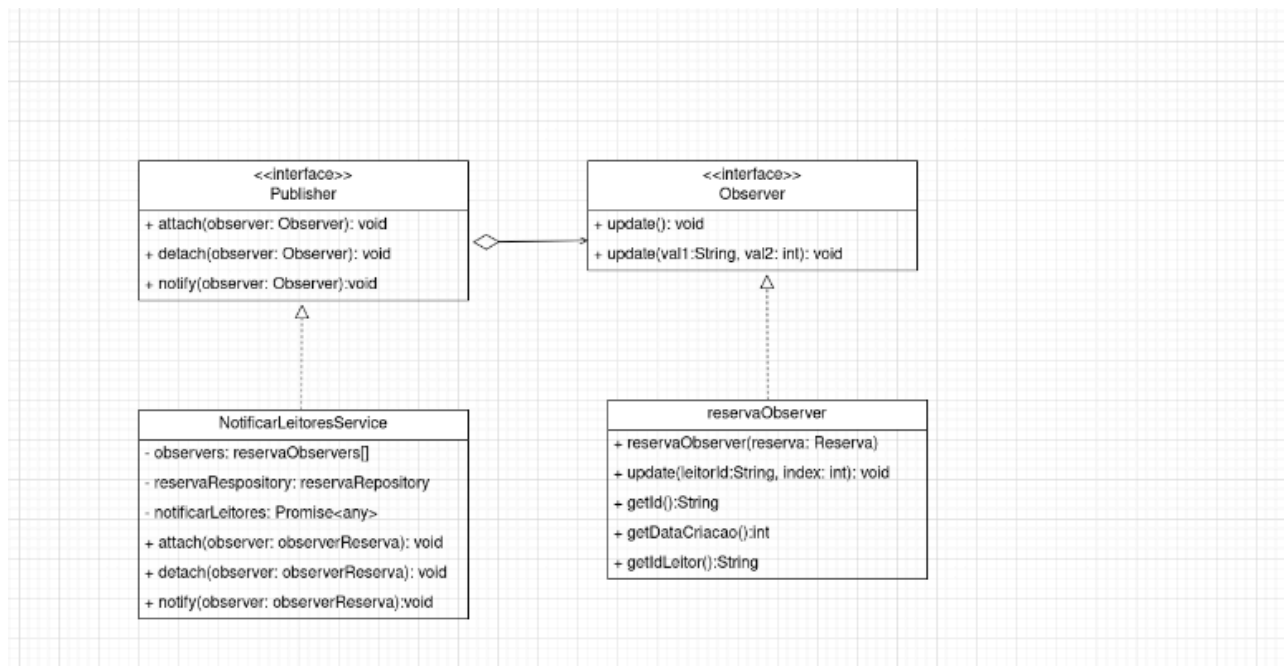
A Reserva no estado Em_Espera observa a Publicação à qual está associada para saber quando um Exemplar torna-se disponível. A Publicação, por sua vez, observa seus Exemplares para identificar mudanças relevantes de estado. Quando um Exemplar passa para o estado Disponível, a Publicação notifica a primeira Reserva da fila, que pode então requisitar a alteração do estado do Exemplar para Reservado. Além disso, outras situações de sincronização entre objetos do domínio podem ser implementadas através da mesma estrutura de observação.

Problema que resolve:

A comunicação entre objetos que precisam reagir a mudanças de estado de outros objetos sem que exista dependência direta entre eles.

Justificativa da escolha:

O fluxo de devolução de um Exemplar envolve Empréstimo, Exemplar, Publicação e Reserva. Nenhuma dessas entidades deveria conhecer todas as demais ou controlar integralmente o processo. O Observer permite que cada entidade seja responsável apenas por suas próprias alterações de estado e notifique os interessados quando necessário, reduzindo o acoplamento e evitando a necessidade de referências diretas entre todos os participantes do fluxo.



State

Contexto de uso:

Os estados de Empréstimo, Exemplar e Reserva podem ser representados por hierarquias de classes ao invés de simples atributos armazenados em tabelas. Cada estado implementa apenas os comportamentos válidos para aquela situação. Por exemplo, um Exemplar em estado Disponível pode ser emprestado ou reservado, enquanto um Exemplar em estado Emprestado pode apenas aguardar devolução. Da mesma forma, uma Reserva em estado Em_Espera possui responsabilidades diferentes de uma Reserva em estado Bloqueante ou Finalizada.

Problema que resolve:

Diferentes comportamentos de uma classe dependendo do estado no qual ela se encontra.

Justificativa da escolha:

Grande parte das regras do domínio depende do estado atual das

entidades. Utilizar apenas atributos de estado tende a concentrar diversas estruturas condicionais em poucas classes. Com State, cada comportamento passa a ficar encapsulado no próprio estado responsável por ele, tornando as transições mais explícitas e reduzindo a complexidade das entidades principais.

Façade

Contexto de uso:

Operações realizadas pelos atores do sistema, como registrar empréstimos, efetuar devoluções, criar reservas ou cancelar reservas, normalmente exigem a coordenação de várias entidades do domínio. Uma fachada pode fornecer métodos de alto nível responsáveis por orquestrar essas operações sem expor todos os detalhes internos para as interfaces gráficas ou controladores.

Problema que resolve:

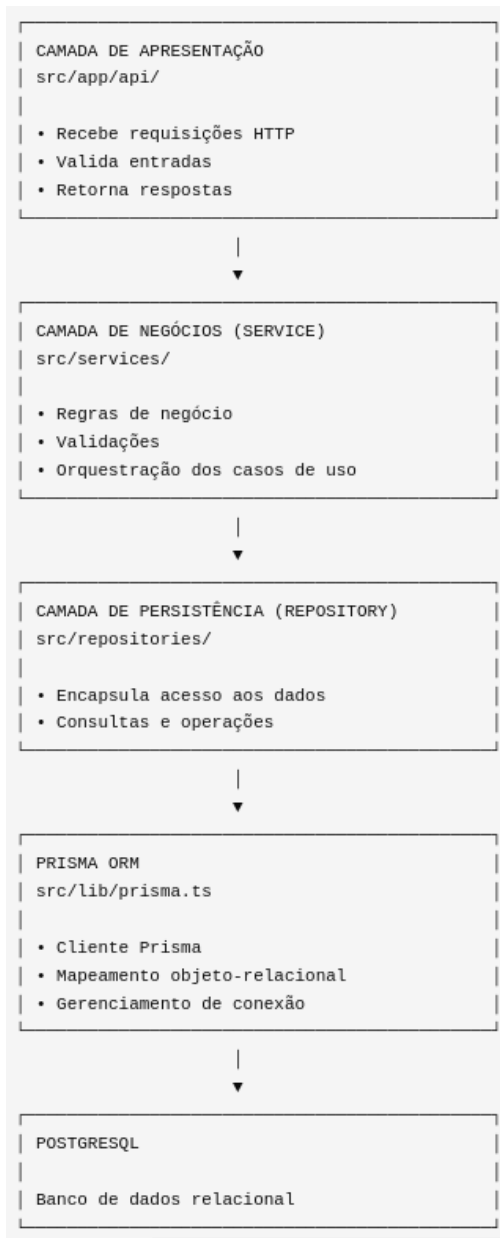
O excesso de dependências entre a camada responsável pelas interações do usuário e as diversas classes de negócio necessárias para executar uma operação completa.

Justificativa da escolha:

Processos como empréstimo e devolução envolvem múltiplas validações e alterações de estado em diferentes objetos. A fachada fornece um ponto único de acesso para essas funcionalidades, reduzindo o acoplamento dos clientes com o modelo de domínio e concentrando a coordenação dos casos de uso em um componente específico. Isso também facilita alterações futuras na implementação interna sem impactar as interfaces que utilizam o sistema.

Arquitetura Geral

O sistema de bibliotecas segue um padrão de arquitetura em camadas, com uma separação baseada em apresentação, negócios, persistência, banco de dados e da interação entre o banco de dados a camada de persistência, um papel desempenhado pelo Prisma ORM.



Caso de Uso de Listagem

Listagem Mídias da Biblioteca: obterMidias(): Promise<Midia[]>

- Obtém uma lista de mídias existentes no sistema

Listagem Exemplares Por Midia: obterExemplaresPorMidia(idMidia: string): Promise<Exemplar[]>

- Obtém uma lista de exemplares existentes de uma mídia, mostrando informações sobre a disponibilidade, por exemplo

Casos de Uso Transacionais

Finalização de Empréstimo:

finalizarEmprestimo(idEmprestimo: string): Promise<Emprestimo>

- Encontra o empréstimo
- Atualiza o empréstimo para finalizado
- Atualiza o exemplar para disponível

Uso de IA

- Os agentes GitHub Copilot, Claude e ChatGPT foram utilizados neste projeto
- Os testes em sua totalidade foram criados por meio do GitHub Copilot
- Claude e ChatGPT foram usados para o tirar dúvidas sobre o funcionamento do framework das tecnologias utilizadas e quanto a implementação dos padrões GOF
- Algumas páginas tiveram funções adicionadas com o uso do GitHub Copilot. A página do formulário dos leitores (LeitorForm.tsx), por exemplo, teve a adição da máscara de CPF e partes do front end criadas desta
- Há páginas neste sistema que foram geradas integralmente com o uso do GitHub Copilot
- O diagrama de camadas foi gerado pelo ChatGPT